The 32-Bit ALU is a combinational processing unit that accepts two 32-bit operands (A and B), a 4-bit opcode, and a carry-in signal (Cin). It produces a 32-bit result and a set of status flags in a single combinational pass — there are no internal registers or pipeline stages.

The ALU supports 16 distinct operations organised into five functional groups: Arithmetic, Shift, Logic, Rotate, and Special/Comparator. Status flags (Z, N, C, V) are updated on every arithmetic, shift, rotate, logic, and pass operation. Dedicated comparator outputs (AequalB, AgreaterB, AlesserB) are driven exclusively by the two comparator opcodes (1101 and 1111).

Port Interface Summary

Signal	Width	Direction	Description
A	32-bit	Input	Primary operand
B	32-bit	Input	Secondary operand
Opcode	4-bit	Input	Selects one of 16 ALU operations
Cin	1-bit	Input	Carry-in for arithmetic operations
Result	32-bit	Output	Computed result
Z	1-bit	Output	Zero flag — asserted when Result = 0
N	1-bit	Output	Negative flag — asserted when Result[31] = 1
C	1-bit	Output	Carry-out flag from adder/shift/rotate
V	1-bit	Output	Overflow flag — valid for signed arithmetic only
AequalB	1-bit	Output	Asserted when A = B (COMP-U / COMP-S only)
AgreaterB	1-bit	Output	Asserted when A > B (COMP-U / COMP-S only)
AlesserB	1-bit	Output	Asserted when A < B (COMP-U / COMP-S only)

Functional Groups

Functional Group	Opcodes	Functions
Arithmetic	0000, 0001, 1000, 1001	ADD (A+B+Cin), SUB (A−B), INCR (A+1), DECR (A−1)
Shift	0010, 0011	SLL – Logical Shift Left by 1; SLR – Logical Shift Right by 1
Logic	0100–0111	AND, OR, XOR (bitwise dyadic); NOT (bitwise complement of A)
Rotate	1010, 1011	ROL – Rotate Left by 1 (wrap MSB→LSB); ROR – Rotate Right by 1 (wrap LSB→MSB)
Special / Comparator	1100–1111	PASS (A), ZERO (0x00000000), COMP-U (unsigned compare), COMP-S (signed compare)

Comparator Truth Table

Condition	AequalB	AgreaterB	AlesserB
A equals B (A = B)	**1**	**0**	**0**
A greater than B (A > B)	**0**	**1**	**0**
A less than B (A < B)	**0**	**0**	**1**

Signed Comparison (COMP-S, Opcode 1111)

Both operands A and B are interpreted as 32-bit two's-complement signed integers. Bit 31 is the sign bit. The magnitude comparison accounts for sign; for example, 0xFFFFFFFF (−1) is less than 0x00000001 (+1). The same mutually exclusive output convention applies. The Result bus outputs A-B.

Unsigned Comparison (COMP-U, Opcode 1101)

Both operands A and B are treated as unsigned 32-bit integers. The full 32-bit magnitudes are compared without sign extension. The output triple {AequalB, AgreaterB, AlesserB} is mutually exclusive — exactly one output is asserted at a time. The result bus outputs A-B.

Functional Table

The table below fully specifies every opcode. Flag columns use the legend defined in the footer row.

Opcode	Mnemonic	Group	Operation	Result Expression	Z	N	C	V	A=B	A>B	A<B
Arithmetic Operations (Opcodes 0000–0001, 1000–1001)											
0000	ADD	Arithmetic	Add A, B and carry-in	`A + B + Cin`	✓	✓	✓	✓	0	0	0
0001	SUB	Arithmetic	Subtract B from A (two's complement)	`A + (~B) + 1`	✓	✓	✓	✓	0	0	0
1000	INCR	Arithmetic	Increment A by 1	`A + 1`	✓	✓	✓	✓	0	0	0
1001	DECR	Arithmetic	Decrement A by 1	`A − 1`	✓	✓	✓	✓	0	0	0
Shift Operations (Opcodes 0010–0011)											
0010	SLL	Shift	Logical shift A left by 1 bit	`A << B; LSB = 0`	✓	✓	0	0	0	0	0
0011	SLR	Shift	Logical shift A right by 1 bit	`A >> B; MSB = 0`	✓	✓	0	0	0	0	0
Logic Operations (Opcodes 0100–0111)											
0100	AND	Logic	Bitwise AND of A and B	`A & B`	✓	✓	0	0	0	0	0
0101	OR	Logic	Bitwise OR of A and B	`A | B`	✓	✓	0	0	0	0	0
0110	XOR	Logic	Bitwise XOR of A and B	`A ^ B`	✓	✓	0	0	0	0	0
0111	NOT	Logic	Bitwise complement of A	`~A`	✓	✓	0	0	0	0	0
Rotate Operations (Opcodes 1010–1011)											
1010	ROL	Rotate	Rotate A left by 1 (MSB→LSB wrap)	`{A[30:0], A[31]}`	✓	✓	0	0	0	0	0
1011	ROR	Rotate	Rotate A right by 1 (LSB→MSB wrap)	`{A[0], A[31:1]}`	✓	✓	0	0	0	0	0
Special / Comparator Operations (Opcodes 1100–1111)											
1100	PASS	Special	Pass operand A unchanged	`A`	✓	✓	0	0	0	0	0
1101	COMP-U	Comparator	Unsigned magnitude comparison of A and B	`Compare |A|, |B|`	✓	✓	0	0	✓	✓	✓
1110	ZERO	Special	Force result to all-zeros	`32'b0`	✓	0	0	0	0	0	0
1111	COMP-S	Comparator	Signed two's-complement comparison of A and B	`Compare A, B (signed)`	✓	✓	0	0	✓	✓	✓
Legend: ✓ = Flag generated and valid | **0** = Flag forced to logic 0 | **–** = Not applicable to this operation											

Module Behaviour Summary

- ARITHMETIC — ADD (0000) produces A + B + Cin; the carry-in is additive and participates in the sum. Z, N, C, and V flags are all valid on completion.
- ARITHMETIC — SUB (0001) performs two's-complement subtraction as A + (~B) + 1; no explicit Cin is required because the one is embedded in the encoding. Z, N, C, and V flags are all valid.
- ARITHMETIC — INCR (1000) and DECR (1001) modify only operand A; operand B is ignored. Carry-out reflects overflow into or from bit 31.
- LOGIC — AND, OR, XOR (0100–0110) perform independent bitwise operations on all 32 bit pairs of A and B. Carry (C) and Overflow (V) are forced to 0 because they have no arithmetic meaning in bitwise operations.
- LOGIC — NOT (0111) complements all 32 bits of A; operand B is ignored. C and V are forced to 0.
- SHIFT — SLL (0010) shifts A left by one position; the vacated LSB is filled with 0 and the ejected MSB is captured in the Carry flag C. SLR (0011) shifts A right by one position; the vacated MSB is filled with 0 and the ejected LSB is captured in C.
- ROTATE — ROL (1010) rotates A left by one bit; the ejected MSB wraps into the LSB position, and C reflects the MSB before rotation. ROR (1011) rotates A right by one bit; the ejected LSB wraps into the MSB position, and C reflects the LSB before rotation.
- COMPARATOR — COMP-U and COMP-S are the only operations that drive AequalB, AgreaterB, and AlesserB.
- COMPARATOR — COMP-U (1101) treats both A and B as unsigned values; COMP-S (1111) treats both as two's-complement signed values. The distinction affects comparison of values with bit 31 set.
- SPECIAL — PASS (1100) routes operand A to the Result bus unchanged. B is ignored; Z and N reflect the value of A; C and V are forced to 0.
- SPECIAL — ZERO (1110) forces Result = 0x00000000 unconditionally, regardless of A and B. Z is asserted (1), N is de-asserted (0), and C and V are forced to 0.
- FLAGS — Z (Zero) is asserted whenever Result[31:0] is all-zeros. N (Negative) mirrors Result[31]. C (Carry) is valid for arithmetic and shift/rotate operations; forced 0 for logic and comparator operations. V (Overflow) is valid only for ADD and SUB; forced 0 for all other operations.
- ALL OUTPUTS — All outputs (Result, flags, comparator signals) are combinational and update within one propagation delay of any change on A, B, Opcode, or Cin. No clock is required.
- OPERAND USAGE — For unary operations (NOT, INCR, DECR, PASS, ZERO, SLL, SLR, ROL, ROR), operand B is a don't-care and is internally gated off. Cin is relevant only to ADD; it is ignored by all other operations.